

AI-Powered FTDC Analyzer for MongoDB

1. Background & Problem Statement

MongoDB ships a built-in monitoring mechanism called FTDC — Full-Time Diagnostic Data Capture. It continuously records detailed server metrics (WiredTiger cache and ticket stats, opcounters, connections, replication lag, system CPU and disk I/O, and much more) into compact binary files inside the server's diagnostic.data directory. These files capture a precise time-series snapshot of everything happening inside the database engine, sampled every second.

When a production incident happens — for example, a sudden drop in WiredTiger read/write tickets causing request queuing and latency spikes — the FTDC data is the most authoritative source of truth for what the server was doing at the time. However, analyzing it today requires:

- Knowing which binary parser tools to use (e.g. ftdc-utils, keyhole, mdiag) to decode the files
- Manually plotting dozens of metrics in Grafana or custom scripts to spot the anomaly window
- Knowing from experience which metrics to correlate (e.g. ticket exhaustion → cache eviction pressure → checkpoint stall → dirty bytes buildup)

There is no tool today that can take raw FTDC data, automatically identify what went wrong, and explain the root cause in plain language.

2. Project Goal

Build a web application with an embedded AI agent that accepts an uploaded FTDC data folder, automatically parses and analyses the metrics, detects anomalies, generates relevant graphs, and iteratively reasons through the data to identify the root cause of the issue — asking the user for additional inputs only when necessary, and only all at once.

The final output should be a clear, accurate root cause report explaining exactly why the issue happened (e.g. why ticket drops occurred), tracing back to the deepest bottleneck, with concrete suggested next steps to resolve it.

3. Core Components

3.1 FTDC File Upload & Parsing

The user uploads a MongoDB diagnostic.data folder through the web UI. The application:

- Accepts the full diagnostic.data directory (multiple binary FTDC files)
- Parses the binary BSON-encoded FTDC format to extract all metric time series
- Organises metrics into structured, queryable data (timestamp → metric name → value)
- Handles both full-metadata documents (sampled every 60s) and delta-compressed documents (sampled every 1s)

Key metric groups extracted: WiredTiger tickets (read/write available & out), WiredTiger cache (bytes in cache, dirty bytes, bytes read/written), opcounters (insert/update/delete/query/getmore), connections (current, available), globalLock queue depths, replication lag, system CPU and disk I/O.

3.2 Anomaly Detection via LLM

Once metrics are parsed, the agent feeds a structured summary of the time series data to an LLM to identify anomaly windows. The LLM is asked to look for:

- Sudden drops in WiredTiger read or write ticket availability (ticket exhaustion)
- Spikes in cache dirty bytes or eviction pressure
- Abnormal latency increases in operation counters
- Unusual connection surges or drops
- Checkpoint stalls visible in WiredTiger cache write-out metrics
- Correlation between multiple metrics degrading simultaneously

The LLM returns the identified anomaly time windows with a brief description of what it observed, which are then used to scope all downstream analysis.

3.3 Automated Graph Generation

For each detected anomaly window, the app automatically generates interactive time-series graphs for the metrics most relevant to that anomaly type. Graphs are rendered in the browser and cover a configurable time window around the anomaly (e.g. 30 minutes before and after). Example graphs generated for a ticket drop anomaly:

Metric	Why It Is Relevant
WT read / write tickets available	Primary signal — shows when and how fast tickets were exhausted
WT cache dirty bytes %	Indicates if eviction pressure was forcing checkpoint activity

WT bytes read into cache	Shows read amplification / cache miss rate
opcounters — all operation types	Identifies if a query surge preceded the ticket drop
Connections current	Reveals if a connection spike drove concurrent ticket demand
globalLock currentQueue	Shows how many operations were queued waiting for the lock
System CPU user + iowait %	Identifies if disk I/O or CPU saturation contributed
Disk read / write bytes	Correlates storage-layer pressure with engine behaviour

3.4 Iterative Root Cause Analysis Agent

This is the core of the project. After anomaly detection and graph generation, the agent begins an iterative reasoning loop to find the root cause:

Step 1 — First-pass analysis

The agent analyses all parsed metrics and the anomaly window together and forms an initial hypothesis about what caused the issue (e.g. “write ticket exhaustion triggered by a spike in concurrent write operations, correlating with elevated cache dirty bytes suggesting checkpoint pressure”).

Step 2 — Deepen the analysis

The agent evaluates whether its current hypothesis is sufficiently explained by the FTDC data alone, or whether more information is needed. It follows a structured priority order for gathering additional evidence:

- **Graylog MCP (if configured):** Query application logs around the anomaly time window to identify which queries or operations were running — looking for slow queries, collection scans, missing indexes, or abnormal query volumes.
- **Profiling data (if provided by user):** If query-level detail is still insufficient, ask the user to provide MongoDB profiler output (db.system.profile) captured during the incident window for deep per-query analysis.

Step 3 — Ask the user — all at once

If the agent determines it needs additional context from the user (e.g. whether a deployment happened, what the server’s memory configuration is, whether profiling was enabled), it collates ALL open questions into a single structured request and presents them to the user at once. It never asks one question, waits, then asks another.

Step 4 — Continue until root cause is confirmed

The agent incorporates user answers and any newly fetched data, refines its hypothesis, and continues reasoning. It repeats until it can confidently state the root cause at the deepest bottleneck level — not just a symptom, but the actual underlying driver (e.g. not just “tickets

were exhausted” but “a query missing an index on collection X caused full collection scans, each holding a read ticket for ~800ms, driving concurrent ticket demand to 128 which exhausted the pool”).

3.6 Final Output Report

At the end of the analysis, the app produces a structured root cause report containing:

- **Anomaly summary:** What happened, when it happened, and which metrics degraded
- **Root cause:** The actual underlying driver of the issue, traced to the deepest bottleneck point, with supporting evidence from FTDC data, query logs, or profiling data
- **Evidence chain:** A step-by-step trace showing how the agent arrived at the root cause (which metrics led to which hypothesis, what data confirmed or ruled it out)
- **Relevant graphs:** All generated graphs embedded in the report for reference
- **Suggested next steps:** Concrete, actionable recommendations to fix the root cause and prevent recurrence

4. End-to-End Agent Workflow

The following describes the full flow from upload to final report:

1. User uploads the MongoDB diagnostic.data folder via the web UI
2. Backend parses all FTDC binary files and extracts structured metric time series
3. LLM analyses the metrics and identifies anomaly windows with initial observations
4. App generates interactive graphs for relevant metrics scoped to the anomaly time window
5. Agent begins iterative root cause reasoning loop — forms hypothesis, evaluates evidence, deepens analysis
6. If query-level data is needed, agent calls Graylog MCP if configured to fetch relevant logs around the anomaly window
7. If the agent needs user input or profiling data, it collates all open questions and asks the user once
8. Agent incorporates new inputs and continues reasoning until root cause is confirmed
9. Final root cause report is generated and displayed in the UI with all supporting evidence and next steps